Avaliação de Desempenho e Programação Paralela com Threads

Guilherme H. Viana Pichitelli Vitor, Pedro H. Bertoni de Souza

Departamento de Informática – Universidade Estadual de Maringá (UEM)
Maringá, PR - Brasil

ra145092@uem.br, ra143345@uem.br

Abstract. *This work presents an analysis of parallel programming performance in C using the POSIX Threads (Pthreads) library. Four algorithms were implemented and evaluated, then performed using different numbers of threads, varying between 1, 2, 4, and 8, considering execution time, speedup, and parallel efficiency.*

Resumo. *Este trabalho apresenta uma análise do desempenho da programação paralela em C utilizando a biblioteca POSIX Threads (Pthreads). Foram implementados e avaliados quatro algoritmos, então realizados com diferentes quantidades de threads, variando entre 1, 2, 4 e 8, considerando o tempo de execução, o speedup e a eficiência paralela.*

1. Introdução

O presente trabalho apresenta uma análise detalhada e quantitativa sobre o desempenho da programação paralela utilizando a biblioteca POSIX Threads (Pthreads) na linguagem C. Foram desenvolvidos e avaliados quatro algoritmos distintos para execução em multithread: processamento de imagens (aplicação de filtro de média 3x3), multiplicação de matrizes, e dois contadores de varredura (com e sem controle de concorrência por mutex).

Além de explicitar o funcionamento interno e as premissas arquiteturais das soluções desenvolvidas, avalia-se o comportamento temporal e a escalabilidade dos algoritmos ao variar a quantidade de threads concorrentes entre 1, 2, 4 e 8 unidades de execução.

Todos os experimentos computacionais foram realizados em um ambiente de testes padronizado, sob um processador Intel Core i5-12500H (arquitetura híbrida de 12 núcleos físicos e 16 threads lógicas) e 24 GB de memória RAM DDR4 operando a 3200 MHz. O ambiente de execução baseou-se no WSL2 (Windows Subsystem for Linux 2) com distribuição Linux sobre o sistema operacional Windows 11. Para assegurar a reprodutibilidade e a estabilidade dos dados de medição, as únicas aplicações ativas no sistema durante as etapas de benchmarking eram o ambiente WSL2, o editor VSCode e uma janela do navegador Microsoft Edge isolada em uma aba de planilha eletrônica para registro inicial dos dados.

2. Pthreads

Pthreads (POSIX Threads) é a biblioteca padrão de threads em C para sistemas Unix/Linux. Uma *thread* é uma linha de execução dentro de um processo. Todas as threads de um mesmo processo compartilham o mesmo espaço de memória (variáveis globais, *heap*, etc.), porém cada uma possui sua própria pilha. O comando de criação de uma thread é padronizado da seguinte forma:

```
C/C++

pthread_create(&threads[i], NULL, trabalho, &tarefas[i]);
```

- `&threads[i]`: Endereço onde o sistema armazena o identificador (`pthread_t`) da thread criada. Esse identificador é necessário posteriormente na chamada de `pthread_join`.
- `NULL`: Atributos de configuração da thread (como tamanho da pilha ou estado detached). Passar `NULL` aplica os valores padrão.
- `trabalho`: Ponteiro para a função a ser executada pela thread. A função deve obrigatoriamente possuir a assinatura `void *f(void *)`.
- `&tarefas[i]`: Argumento passado para a função da thread. Como a assinatura aceita apenas um parâmetro do tipo `void *`, os dados necessários são tipicamente encapsulados em uma estrutura (ex.: `Tarefa`) cujo endereço é fornecido. Este `struct Tarefa` atua como um pacote de parâmetros para a thread. Como o `pthread_create` aceita um único argumento genérico `void *`, agrupa-se todas as informações essenciais na estrutura e passa-se seu endereço. Dessa forma, cada thread recebe sua própria instância de `Tarefa`, determinando o trabalho específico e a fração dos dados a ser processada.

Após a criação das threads, construa-se uma iteração que abrange todas as threads criadas:

```
for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(threads[i], NULL);
```

Este comando bloqueia a função Main de continuar até que todas as threads sejam concluídas. Além disso, o join libera os recursos que a thread estava utilizando após o término da mesma.

3. Processamento de Imagens

3.1. Conversor de imagem para matrizes

As imagens disponibilizadas precisavam ser transformadas em matrizes para a posterior aplicação do filtro. Para isso, foi desenvolvida uma ferramenta em Python que recebe como entrada uma imagem em escala de cinza de formato quadrado e gera como saída um arquivo .txt contendo a matriz correspondente.

O script em questão, denominado “imgToMat.py”, encontra-se no diretório imgMat. Foram utilizadas as bibliotecas Pillow e NumPy. De modo geral, o programa carrega o arquivo de imagem .png, converte a estrutura em uma matriz por meio da função np.array e, por fim, salva o resultado em .txt com a função np.savetxt.

De modo análogo, também foi desenvolvido o script em Python “matToImg.py”, que recebe o arquivo .txt com a matriz resultante da aplicação do filtro e a reconverte em uma imagem .png.

3.2. Funcionamento geral do código

O fluxo geral é composto pelas seguintes etapas: (i) leitura da matriz proveniente da imagem; (ii) preparação dos vetores de entrada e saída; (iii) divisão do trabalho entre as threads; (iv) execução paralela do filtro; (v) medição do tempo; e (vi) gravação do resultado.

A imagem é uma matriz quadrada de ordem n, armazenada em um vetor unidimensional alocado dinamicamente. Os elementos são dispostos linha após linha, de modo que o pixel da linha i e coluna j corresponde à posição i * n + j do vetor. Essa representação mantém os dados contíguos na memória, simplifica a alocação e permite que todas as threads acessem a mesma imagem por meio de ponteiros, sem necessidade de cópias.

A função le_matriz realiza a leitura em duas passadas. Na primeira, o arquivo é percorrido apenas para contar a quantidade total de valores. A ordem da matriz é obtida pela raiz quadrada desse total, e o programa verifica se o resultado é um inteiro exato, o que garante que a matriz é quadrada. Na segunda passada, o ponteiro do arquivo é reposicionado no início (rewind) e os valores são armazenados no vetor. Esse método permite que o programa processe matrizes de qualquer ordem (como 256×256 ou 1024×1024) sem alterar o código-fonte, além de tratar erros de abertura de arquivo, formato inválido e falta de memória.

Para cada pixel interno da imagem, o novo valor é a média inteira dos nove valores da sua vizinhança 3×3, incluindo o próprio pixel. Os pixels das bordas (primeira e última linha e coluna) permanecem inalterados, pois não possuem vizinhança completa. O funcionamento da função aplicaFiltro é explorada mais minuciosamente posteriormente neste relatório.

O processamento é dividido entre NUM_THREADS = X threads. As n − 2 linhas internas são distribuídas em faixas contíguas: cada thread recebe (n − 2) / X linhas e as linhas restantes são repartidas, uma a uma, entre as primeiras threads, de modo que a diferença de carga entre quaisquer duas threads seja de no máximo uma linha.

Como a função de uma thread aceita apenas um argumento, as informações necessárias são agrupadas na estrutura Tarefa, que contém o identificador da thread, o intervalo de linhas [inicio, fim), a ordem n e os ponteiros para os vetores de entrada e saída. Cada thread é criada com pthread_create e executa a função aplicaFiltro sobre a sua faixa. A thread principal aguarda o término de todas por meio de pthread_join.

O tempo de execução é medido com clock_gettime utilizando o relógio CLOCK_MONOTONIC, que não é afetado por ajustes no relógio do sistema. A medição é feita imediatamente antes da criação das threads e imediatamente após o último pthread_join, de modo que o valor obtido corresponde exclusivamente à etapa de processamento paralelo, desconsiderando o tempo de leitura e de escrita em arquivo.

Ao final, a função escreve_matriz grava a imagem filtrada em saida.txt (o nome do arquivo de saída pode ser alterado diretamente no código), com os valores de cada linha separados por espaço. Em seguida, a memória alocada dinamicamente é liberada. O programa também exibe no terminal a ordem da matriz, os valores das posições [0][0] e [n−1][n−1] e o tempo de execução.

3.3. Filtro de média 3x3.

A imagem é uma matriz n × n guardada num vetor 1D, linha após linha. O elemento (i, j) fica em vetor[i * n + j]. A função é executada por cada thread, e cada thread recebe um pedaço diferente das linhas da imagem.

Ao analisarmos a assinatura da função `void *aplicaFiltro(void *arg)`, perceba que o argumento é genérico, pois o pthread_create exige que a função da thread receba um *void* * e devolva um *void* *.

Através da primeira linha da função: `Tarefa *t = (Tarefa *)arg;`, converte-se o void * de volta para o tipo real, o Tarefa *. A struct Tarefa contém tudo de que a thread precisa.

Em seguida, copia-se n para uma variável local com o objetivo de facilitar a escrita do algoritmo.

Após a preparação inicial, finalmente iniciam-se os laços de repetição. Na primeira camada temos: `for (int i = t->inicio; i < t->fim; i++)`. Este é o laço das linhas. Percorre apenas as linhas da thread. Como o main distribui início e fim entre 1 e n-2, a primeira e a última linha da imagem nunca são processadas, o que preserva as bordas de cima e de baixo. Em seguida, temos o próximo laço de repetição: `for (int j = 1; j < n - 1; j++)`. É o laço das colunas. Começa em 1 e termina em n-2, ou seja, ignora a primeira e a última coluna. Isso preserva as bordas da esquerda e da direita. Além disso, garante que o acesso à vizinhança (j-1 e j+1) nunca saia da matriz.

Na linha seguinte, temos o int n = 0; Esta linha zera o acumulador para cada novo pixel que será processado;

Os laços:

```C/C++
for (int di = -1; di <= 1; di++)
      for (int dj = -1; dj <= 1; dj++)
            soma += t->entrada[(i + di) * n + (j + dj)];
```

Percorre a vizinhança 3 × 3 em torno do pixel (i, j). di e dj são os deslocamentos de linha e coluna, cada um indo de -1 a +1, o que dá 9 combinações (Tabela 1). A expressão (i + di) * n + (j + dj) converte a posição 2D do vizinho para o índice no vetor 1D. Note que o pixel central também entra na soma, como manda o filtro de média 3 X 3

Tabela 1: Vizinhança 3x3 para aplicação do filtro de média.

(i-1, j-1)	(i-1, j)	(i-1, j+1)
(i, j-1)	(i, j)	(i, j+1)
(i+1, j-1)	(i+1, j)	(i+1, j+1)

Em seguida, temos a linha "*t->saida[i * n + j] = soma / 9;*" que divide a soma dos 9 valores por 9 e grava o resultado na posição (i, j) do vetor de saída. É uma divisão inteira, então a parte decimal é descartada (truncada). Por fim, a thread termina com a linha return NULL;

3.4. Resultados

Os resultados obtidos nas medições de tempo de execução do filtro de média 3x3 estão sintetizados na Tabela 2. Para fins de avaliação de desempenho e inferência de comportamento temporal, considerou-se a média aritmética dos tempos observados ao longo de cinco execuções para cada cenário experimental. Além disso, calculamos o speedup que é a relação entre tempo sequencial e tempo paralelo e, também, a eficiência paralela que indica o aproveitamento das threads:

$$Speedup = \frac{Tempo\ Sequencial}{Tempo\ Paralelo}$$

$$Eficiencia\ Paralela\ (EP) = \frac{Speedup}{Quantidade\ de\ Threads}$$

Conforme o esperado hipotéticamente, a elevação do grau de paralelismo via aumento da quantidade de threads concorrentes resultou em uma redução progressiva do tempo total de processamento. No entanto, em termos de latência absoluta percebida pelo usuário, as variações temporais situam-se na ordem de milissegundos e frações submilissegundas, tornando a ganho computacional imperceptível em tempo real sem instrumentação adequada.

Em termos de aceleração (speedup), a transição de 1 para 2 threads proporcionou a variação de desempenho mais expressiva em todas as dimensões de matriz testadas. Para imagens de menor dimensão espacial (256x256 pixels), o ganho relativo ao transicionar de 2 para 4 threads mostrou-se moderado; contudo, a escalabilidade ao atingir 8 threads demonstrou saturação, resultando em ganhos marginais praticamente nulos. Essa estabilização é corroborada, por exemplo, pela convergência de tempos verificada na terceira rodada de testes entre as configurações de 4 e 8 threads, que apresentaram tempo de 0.000566 segundos e 0.000647 segundos respectivamente.

Por outro lado, ao analisar as dimensões intermediárias e superiores (512x512 e 1024x1024 pixels, respectivamente), a elevação de 4 para 8 threads gerou uma redução mais significativa no tempo médio de execução em comparação com a imagem de

ordem inferior. Tal comportamento evidencia que o overhead de gerenciamento e sincronização das threads é diluído à medida que a carga de trabalho computacional por thread cresce, demonstrando maior eficiência de escalabilidade em matrizes de maior magnitude.

Entretanto, o padrão se repete independentemente do tamanho da imagem. Tal fato pode ser observado, por exemplo, ao observarmos que a eficiência paralela diminui conforme mais threads são adicionadas.

Desse modo concluímos que: Quanto mais threads, mais rápido (para este problema), porém só até certo ponto. O ganho de eficiência ao adicionar threads depende também do tamanho do problema, ou seja, quanto maior o problema, o ganho de eficiência com mais threads é maior se comparado a um problema menor.

Tabela 2: dados extraídos do experimento de filtro 3x3.

EXPERIMENTO	Média (Seg)	Desvio Padrão (Seg)	Speedup	EP
img 256 1 thread	0,000964	0,000023	N/A	N/A
img 256 2 thread	0,000631	0,000034	1,53	0,765
img 256 4 thread	0,000529	0,000093	1,82	0,455
img 256 8 thread	0,000525	0,000069	1,84	0,230
img 512 1 thread	0,003659	0,000104	N/A	N/A
img 512 2 thread	0,002021	0,000072	1,81	0,905
img 512 4 thread	0,001571	0,000280	2,32	0,580
img 512 8 thread	0,001258	0,000187	2,9	0,362
img 1024 1 thread	0,013323	0,000583	N/A	N/A
img 1024 2 thread	0,006741	0,000446	1,98	0,990
img 1024 4 thread	0,003866	0,000502	3,45	0,862
img 1024 8 thread	0,002976	0,000256	4,47	0,559

3.5. Executando

Para executar o “imgToMat.py”, vá até a raiz do projeto e utilize o seguinte comando no terminal para ativar o ambiente venv:

```
Shell

source .venv/bin/activate
```

Volte ao diretório em que o programa python se encontra. De preferência, garanta que a imagem png que deseja transformar em matriz esteja no mesmo diretório do arquivo python. Informe o nome do arquivo diretamente no código python e execute o arquivo.

No arquivo “filter.c” você pode escolher a quantidade de threads que deseja criar e, na função main, escolha o nome do arquivo .txt que contenha a matriz. No terminal, compile com:

```
C/C++

gcc arquivo.c -o arquivo -lm -pthread
```

A saída será um arquivo .txt contendo a matriz resultante, você pode escolher o nome do arquivo de saída também na função main.

Por último, para transformar a matriz resultante numa nova imagem, execute o arquivo “matToImg.py”. Lembre-se de informar o nome do arquivo no código python e que o ambiente virtual venv deve estar ativo.

4. Multiplicação de Matrizes

4.1. Funcionamento Geral do código

Para o cálculo de multiplicação entre matrizes, é necessário percorrer uma linha da primeira matriz (A) multiplicando cada elemento pelos na coluna da segunda matriz (B) e armazenando a soma desses produtos na posição respectiva àquela linha e coluna, assim passando pelas “i”s linhas de A e “j”s colunas de B, completando uma matriz resultante (C) i x j.

Para divisão de atividade para as threads, cada uma tem uma “fatia” das linhas da matriz C para ser montada, onde com o tamanho total cada uma recebe a mesma proporção em ordem, com possibilidade de ter 1 linha a mais em certas threads. Informação indicada na `struct Tarefa` como inicio e fim.

É interessante ressaltar fatores como as matrizes A e B são representadas como um vetor 1D para int, pois apresenta valores pequenos, porém para melhor precaução, a matriz C é representada como um vetor 1D para long long, considerando o grande volume de produtos somados. Além disso, a construção, alocação e leitura das matrizes é colocada externamente à cronometragem de tempo de execução do programa.

Considerando o mesmo acesso de matrizes realizado no Processamento de imagens anterior, cada thread percorre as matrizes de entrada A e B (vetores de inteiros) com `[i * n + k]` e `[k * n + j]`, respectivamente, pegando seus elementos e armazenando em `soma`.

```C/C++
for (int i = t->inicio; i < t->fim; i++) {
    for (int j = 0; j < n; j++) {
        long long soma = 0;
        for (int k = 0; k < n; k++) {
            soma += (long long)t->A[i * n + k] * t->B[k * n + j];
        }
        t->C[i * n + j] = soma;
    }
}
```

4.2. Resultados

Extraindo as informações da Tabela 3, fica evidente o ganho em eficiência pelo aumento de Threads no programa, sendo mais perceptível em grandes matrizes, para os exemplos apresentados esse ganho também atingiu valores de milissegundos somente, em maiores escalas essa melhora se torna mais necessária.

Tendo em vista esse ganho, é concluído que, para realização de um mesmo algoritmo repetidas vezes, sem dependência qualquer, a separação por threads torna-se expressiva, onde um mesmo trabalho pode ser fragmentado e realizado simultaneamente sem obstáculos, apenas um ganho direto. É intrigante notar que, essa divisão do trabalho pela metade também, aproximadamente, diminui a média de execução pela metade.

Tabela 3: Dados extraídos do experimento de multiplicação de matrizes.

EXPERIMENTO	Média (Seg)	Desvio Padrão	Speedup	EP
matriz 256 1 thread	0,028588	0,001741	N/A	N/A
matriz 256 2 thread	0,015143	0,000573	1,89	0,944
matriz 256 4 thread	0,008714	0,001355	3,28	0,820
matriz 256 8 thread	0,006634	0,001148	4,31	0,539
matriz 512 1 thread	0,284795	0,007226	N/A	N/A
matriz 512 2 thread	0,141025	0,011738	2,02	1,010
matriz 512 4 thread	0,076613	0,002266	3,72	0,929
matriz 512 8 thread	0,051789	0,004643	5,50	0,687
matriz 1024 1 thread	2,926945	0,048958	N/A	N/A
matriz 1024 2 thread	1,481238	0,029534	1,98	0,988
matriz 1024 4 thread	0,805438	0,031259	3,63	0,908
matriz 1024 8 thread	0,586256	0,017237	4,99	0,624

4.3. Executando

Para a geração das matrizes a serem multiplicadas, foi utilizado o código disponibilizado junto à especificação do trabalho, dessa forma é necessário que somente matrizes quadradas e de mesma ordem sejam enviadas na compilação, caso contrário haverá retorno de um erro.

Considerando a mesma compilação de antes, mas no executável informando os nomes das matrizes a serem multiplicadas e o nome da matriz de saída resultante:

```shell
gcc arquivo.c -o arquivo -lm -pthread


./arquivo matriz_a.txt matriz_b.txt saida.txt
```

5. Contador Com Mutex

5.1. Funcionamento geral do código

Este programa divide o trabalho de contar até 100’000’000 em X threads, de modo que todas elas incrementem a mesma variável acumulador. No final da execução, o valor acumulado deve ser exatamente cem milhões.

Primeiro, analisaremos a struct tarefa que contém o id (identificador de thread), iterações (quantos incrementos devem ser feitos por esta thread), acumulador (ponteiro para a variável compartilhada de modo que todas escrevam no mesmo lugar) e o mutex (ponteiro para o mesmo mutex, também compartilhado).

```c
int iteracoes_totais = 100000000 / NUM_THREADS;
int iteracoes_resto = 100000000 % NUM_THREADS;
```

Cada thread receberá a parte inteira da divisão como quantidade de incrementos. Um valor de threads igual ao resto da divisão receberá uma iteração a mais que as demais threads.

Em seguida, as threads são criadas e disparadas. A função contador só faz um laço de `iteracoes` vezes, incrementando `*t->acumulador` dentro de uma região protegida pelo mutex.

O problema central é que o comando `(*acumulador)++` parece uma única operação, mas na CPU são três:

1. Ler o valor da memória para um registrador
2. Somar 1
3. Escrever o resultado de volta na memória

Tabela 4: Passo a passo da ocorrência de condição de corrida entre duas threads.

Passo	Thread A	Thread B	Valor na memória
1	lê 10		10
2		lê 10	10
3	soma → 11		10
4		soma → 11	10
5	escreve 11		11
6		escreve 11	11

Na tabela 4, podemos observar que dois incrementos aconteceram, mas o valor subiu só 1. Isso é uma *atualização perdida*. Repetido milhões de vezes, o resultado final ficaria bem abaixo de 100.000.000 e mudaria a cada execução.

Para evitar que isso aconteça, utiliza-se o mutex. que garante que só uma thread por vez execute o trecho "ler, somar, escrever", o mutex impede a intercalação do exemplo da tabela 4. O resultado fica sempre correto: 100000000. Num exemplo prático, o mutex serve como uma chave para acessar o comando `(*t->acumulador)++;` utilizando os comandos lock (pega a chave para executar o incremento, se outra thread já estiver executando, espera) e o unlock (devolve a chave)

5.2. Resultados com Mutex

Devido à intensa competição entre as threads pelo acesso ao mutex, observa-se que a execução paralela apresenta um tempo de processamento consideravelmente superior ao da abordagem sequencial. Conforme evidenciado na Tabela X, a eficiência paralela (EP) sofre uma degradação expressiva à medida que novos fluxos de execução são adicionados.

Esse comportamento era teoricamente esperado. Como as operações de bloqueio e desbloqueio (lock e unlock) ocorrem a cada incremento individual (totalizando 100 milhões de chamadas), as threads permanecem predominantemente em estado de disputa pela seção crítica. Na prática, a versão concorrente com 2 threads torna-se mais lenta que a sequencial porque o custo computacional associado à sincronização suplanta significativamente a carga de trabalho útil (uma simples instrução de incremento).

Contudo, nota-se que o tempo médio de execução com 4 threads foi ligeiramente inferior ao obtido com 2 threads. Embora não seja possível determinar a causa exata com absoluta precisão, certas particularidades do experimento auxiliam na interpretação do fenômeno. O desvio padrão registrado com 2 threads (0,177) é o mais elevado do conjunto de dados, sugerindo maior oscilação no escalonamento e alocação das threads pelo sistema operacional, variando entre núcleos físicos distintos, threads lógicas compartilhadas (Hyper-Threading) ou alternâncias de contexto que impactam a coerência do cache. Em contrapartida, com 4 threads, a distribuição de carga pelo

escalonador diminui essa variação estocástica, resultando em um desempenho temporal mais estável (desvio padrão de 0,117).

Tabela 5: Dados extraídos do experimento do contador com mutex.

EXPERIMENTO	Média (Seg)	Desvio Padrão (Seg)	Speedup	EP
contadorMutex 1t	1,251113	0,021407	N/A	N/A
contadorMutex 2t	4,823704	0,177409	0,26	0,130
contadorMutex 4t	4,254707	0,117072	0,29	0,072
contadorMutex 8t	6,670753	0,096369	0,19	0,024

5.3. Executando

Para executar o contador com Mutex, basta compilar o programa utilizando o comando:

```
Shell

gcc arquivo.c -o arquivo -lm -pthread
```

E executar o programa com;

```
Shell

./arquivo
```

6. Contador Sem Mutex

6.1. Funcionamento geral do código

Semelhante ao anterior, este programa divide o trabalho de contar até 100’000’000 em X threads, porém de modo que cada uma apresenta sua variável de contagem própria. No final da execução, a soma das contagens deve ser exatamente cem milhões.

Considerando a mesma `struct` anterior de Tarefa, porém sem a necessidade do mutex, cada thread realiza uma função de contagem com uma variável `contador_local` instanciada com a quantidade de `iteracoes` demandadas. Essa demanda é ajustada de acordo com uma proporção igual que ao juntar todas as contagens resulte no total, por exemplo, para 4 threads contarem até 100’000’000, cada uma apresenta uma contagem única até 25’000’000, pela mesma divisão de iterações totais mostradas no Contador Com Mutex.

6.2. Resultados sem Mutex

Considerando uma independência mais bem definida entre as threads que a execução com mutex, a perda de tempo por execução em múltiplas threads inverte-se, onde por não ter mais um tempo de espera, esperando a permissão de contagem passada pela thread anterior, cada uma realiza sua contagem simultaneamente. Desse modo, há um ganho considerável, em escala de milissegundos, de eficiência, mostrado na Tabela 6.

Esse ganho torna-se menos perceptível com aumento de threads de 4 para 8, porém nas primeiras instâncias de paralelidade de execução essa diminuição na média ocorre quase que pela metade.

Com isso, é evidenciado, novamente, que um trabalho repetitivo pode ser muito bem separado visando um ganho computacional, com a possibilidade de independência entre cada thread.

Tabela 6: Dados extraídos do experimento do contador sem mutex.

EXPERIMENTO	Média (Seg)	Desvio Padrão (Seg)	Speedup	EP
contador 1 thread	0,047759	0,000537	N/A	N/A
contador 2 thread	0,025945	0,001121	1,84	0,920
contador 4 thread	0,015183	0,001886	3,15	0,787
contador 8 thread	0,010182	0,000623	4,69	0,586

6.3. Executando

Para execução e compilação do programa segue o mesmo padrão:

```
Shell

gcc arquivo.c -o arquivo -lm -pthread


./arquivo
```

É retornado no terminal o valor total para confirmação do resultado chegando em 100’000’000.

7. Conclusão

Analisando os tempos de execução, apesar de serem imperceptíveis no nosso tempo humano por serem experimentos de pequena escala, ficou evidente que para maiores escalas o ganho computacional depende da situação. Onde, para casos de atividade repetitiva sem dependência, como no processamento de imagens, multiplicação de matrizes, e um contador sem mutex ocorre um grande ganho. Porém para casos com dependência, como no teste do contador com mutex, acontece uma perda considerável onde as threads disputam pelo mesmo recurso de armazenamento.

Em suma, é necessário ter-se uma análise da escala que está sendo trabalhada e se ocorrerá disputa entre threads para elas serem realmente uma boa decisão arquitetural numa implementação, sendo, na maioria das vezes, ótima.

8. Referências

Linux man-pages project. (s.d.) “pthreads(7) — POSIX threads”, Linux manual page. https://man7.org/linux/man-pages/man7/pthreads.7.html.